

PROFESSIONAL STUDY FLASHCARDS

30 High-Yield, Scenario-Based Cards for Advanced Cloud Architects & DevOps Engineers

Author Block: Generated programmatically by Gemini Notebook for AWS Cloud Architect preparation.

Focus: Advanced AWS CDK v2 commands, lifecycle, security, testing paradigms, and real-world zero-downtime DB migrations.

STUDY ROADMAP & PREPARATION GUIDE

This flashcard deck contains 30 professionally typeset, scenario-based review cards designed specifically to replicate senior-level AWS interview loops. Each card is structured to show conceptual mechanics (**What & Why**) combined with practical code implementation (**How**). Study these cards to master the operational commands introduced in modern 2026 CDK workflows, such as *cdk refactor*, *cdk orphan*, and programmatic *integ-tests*.

CARD 1: CONSTRUCT LEVELS (L1, L2, L3)**FRONT: QUESTION / SCENARIO**

What is the structural difference between Level 1 (L1), Level 2 (L2), and Level 3 (L3) constructs in the AWS CDK?

BACK: MECHANICAL EXPLANATION

- **Level 1 (L1) Constructs**: Low-level "CFN resources" that map 1:1 directly to CloudFormation resource specifications. They have the `Cfn`` prefix (e.g., `CfnBucket``) and require manual configuration of all resource properties.
- **Level 2 (L2) Constructs**: Thoughtfully curated components wrapping L1 resources with intent-based APIs. They include secure defaults, automatically generate boilerplate code and IAM policies, and provide helper methods (e.g., `s3.Bucket``).
- **Level 3 (L3) Constructs**: High-level architectural "patterns" combining multiple resources to achieve a specific task (e.g., `ecsPatterns.ApplicationLoadBalancedFargateService``). They offer highly opinionated enterprise configurations.

CARD 2: CDK COMPILATION & LIFECYCLE (JSII)**FRONT: QUESTION / SCENARIO**

Why does the AWS CDK require a Node.js runtime even when you are writing your infrastructure codebase in Python, Java, C#, or Go?

BACK: MECHANICAL EXPLANATION

The core AWS CDK engine and Construct Library are natively implemented in TypeScript and run on Node.js. To provide polyglot support, the CDK team built **JSII**, an interoperability bridge. JSII allows general-purpose languages to interact with the underlying Node.js runtime. At synthesis time, your native code compiles and executes through a JSII host process that runs the Node.js CDK libraries to serialize your code down to CloudFormation templates.

CARD 3: CDK REFACTOR COMMAND**FRONT: QUESTION / SCENARIO**

When and why would you use the ``cdk refactor`` command, and what are its preview and structural constraints?

BACK: MECHANICAL EXPLANATION

- **When/Why**: Use ``cdk refactor`` when renaming constructs, moving resources between stacks, or restructuring your codebase. By analyzing drift between your code and the deployed state, it generates resource mapping instructions for CloudFormation's refactoring API, updating resource logical IDs without physical deletion.
- **Constraints**: Requires the ``--unstable=refactor`` flag in preview. Refactoring operations must be performed alone; if any actual resource additions, deletions, or property modifications are detected in the template, the command is rejected to prevent accidental side effects.

CARD 4: CDK ORPHAN COMMAND**FRONT: QUESTION / SCENARIO**

How does the ``cdk orphan`` command work, and how does it protect stateful resources during construct upgrades?

BACK: MECHANICAL EXPLANATION

- **Function**: It detaches a physical resource from CloudFormation stack management without deleting it, allowing safe construct migrations (e.g., upgrading an obsolete DynamoDB ``Table`` to ``TableV2``) with zero downtime.
- **Mechanism**: It performs three deployments:
 1. **Resolve**: Resolves all cross-resource references (Ref, GetAtt) to their literal values.
 2. **Decouple**: Sets DeletionPolicy to ``Retain`` and removes ``DependsOn`` tags to isolate the target resource.
 3. **Remove**: Removes the resource from the template entirely while leaving the running physical resource intact.
- **Requirement**: Requires the ``--unstable=orphan`` flag and environment bootstrap version 32+.

CARD 5: ZERO-DOWNTIME CONSTRUCT UPGRADE WORKFLOW

FRONT: QUESTION / SCENARIO

Outline the step-by-step CLI and code workflow to upgrade a live, high-traffic DynamoDB table from the legacy `Table` construct to the modern `TableV2` construct with zero downtime.

BACK: MECHANICAL EXPLANATION

1. **Orphan**: Run `cdk orphan MyStack/OrdersTable --unstable=orphan`. CloudFormation detaches the table and outputs a JSON mapping.
2. **Code Update**: Replace `dynamodb.Table` with `dynamodb.TableV2` in your CDK code, maintaining the same table name.
3. **Import**: Run `cdk import MyStack --resource-mapping-inline '{"OrdersTable794EDED1":{"TableName":"OrdersTable"}}'` using the mapping from step 1.
4. **Deploy**: CloudFormation imports the running table back under managed control. Accept the prompt to deploy and reconcile minor template configurations.

CARD 6: CDK GARBAGE COLLECTION

FRONT: QUESTION / SCENARIO

Explain how `cdk gc` works to reduce S3 and ECR storage costs, and clarify the roles of `rollback-buffer-days` and `created-buffer-days`.

BACK: MECHANICAL EXPLANATION

- **Command**: `cdk gc --unstable=gc` audits S3 asset buckets and ECR registries created during bootstrapping to delete unreferenced build artifacts.
- **Parameters**:
 - `--rollback-buffer-days`: Specifies the number of days an asset must remain isolated (unreferenced) before it becomes eligible for deletion. It protects templates that are needed for pipeline rollbacks.
 - `--created-buffer-days`: Sets the minimum age of an asset before it can be garbage collected, protecting recently built files from race conditions.

CARD 7: CDK PUBLISH-ASSETS**FRONT: QUESTION / SCENARIO**

What is the purpose of the `cdk publish-assets` command, and how is it integrated into professional CI/CD pipelines?

BACK: MECHANICAL EXPLANATION

- **Command**: `cdk publish-assets MyStack --unstable=publish-assets` compiles, bundles, and uploads local files and Docker images to S3/ECR destinations without performing a deployment.
- **Use Case**: It decouples asset compilation and publishing from stack provisioning in CI/CD pipelines. This ensures that all deployment artifacts are securely validated, built, and staged in AWS before starting the CloudFormation stack execution phase, reducing pipeline failures.

CARD 8: STANDALONE TEMPLATE VALIDATION & CONFIGURATION**FRONT: QUESTION / SCENARIO**

What is the purpose of the `cdk validate` command, and how do you configure your CDK application to fail builds immediately on validation violations?

BACK: MECHANICAL EXPLANATION

- **`cdk validate`**: An experimental CLI command that runs policy-as-code and compliance validation plugins against synthesized templates as a standalone step, without triggering a deployment. It can run in `--online` mode to fetch live remote rules.
- **Immediate Failure**: Set the context key `"failSynthOnValidationErrors": true` in `cdk.json`. This tells the CDK compiler to immediately fail synthesis, print a clean validation report, and throw a non-zero exit code if any policy violations are discovered during compliance checks.

CARD 9: CROSS-STACK REFERENCES AND WEAK BINDINGS**FRONT: QUESTION / SCENARIO**

Explain the "deadly embrace" in CDK cross-stack references and how modern CDK features solve it using reference strength controls.

BACK: MECHANICAL EXPLANATION

- **Problem**: Passing a resource attribute from Stack A to Stack B creates a strong export (`Fn::ImportValue`). This locks Stack A's template; you cannot modify or delete the referenced resource in Stack A because Stack B depends on it.
- **Solution**: Enable weak cross-stack references. Configure `"@aws-cdk/core:defaultCrossStackReferences": "weak"` in `cdk.json`. This compiles resource attributes through looser parameter bindings or weaker references, enabling independent resource lifecycle updates.

CARD 10: THE JUNE 2026 NODE.JS 24 BREAKING CHANGE**FRONT: QUESTION / SCENARIO**

What breaking change occurred in June 2026 for CDK Lambda deployments referencing `Runtime.NODEJS_LATEST`, and what is the mitigation?

BACK: MECHANICAL EXPLANATION

- **Breaking Change**: In June 2026, `Runtime.NODEJS_LATEST` resolved to Node.js 24.x. This runtime deprecates support for classic callback-style handler signatures (`exports.handler = function(event, context, callback)`). Pipelines using `NODEJS_LATEST` with callback-based functions broke immediately.
- **Mitigation**: Update all legacy Lambda handlers to modern asynchronous signatures (`async/await`) or explicitly pin the runtime property to a stable version such as `Runtime.NODEJS_22_X`.

CARD 11: CVE-2026-11417 OS COMMAND INJECTION VULNERABILITY**FRONT: QUESTION / SCENARIO**

Describe the OS Command Injection vulnerability (CVE-2026-11417) identified in the CDK local bundling pipeline and its architectural remediation.

BACK: MECHANICAL EXPLANATION

- **Vulnerability**: In `aws-cdk-lib` versions prior to 2.245.0 (2.246.0 on Windows), the `NodejsFunction` local bundling process failed to properly escape bundling properties (e.g. `externalModules`, `esbuildArgs`). An actor controlling these properties in the CDK application could inject shell metacharacters to execute arbitrary shell commands on the build machine.
- **Remediation**: Upgrade `aws-cdk-lib` to a patched version. The remediation integrates robust parameter escaping (such as `posixShellEscape`) inside the Docker bundling scripts.

CARD 12: ASYNCHRONOUS INTEGRATION TESTING PHASES**FRONT: QUESTION / SCENARIO**

How do you structure an asynchronous integration test inside `@aws-cdk/integ-tests-alpha` to assert against eventual consistency?

BACK: MECHANICAL EXPLANATION

Since cloud resource states are eventually consistent (e.g., waiting for SQS to trigger Lambda and write to DynamoDB), tests cannot assert immediately. `Integ-tests-alpha` supports API chaining and polling:

1. **Trigger**: Invoke the action under test (e.g., publish an SNS message using `integ.assertions.awsApiCall('SNS', 'publish')`).
2. **Chain**: Use `.next()` to bind a subsequent assertion (e.g., `integ.assertions.awsApiCall('DynamoDB', 'getItem')`).
3. **Poll**: Append `.expect(...)` and configure `.waitForAssertions()` to poll the target resource at specified intervals (e.g. every 10s) until the assertions match or timeout occurs.

CARD 13: RUNNING INTEGRATION TESTS LOCALLY (LOCALSTACK)**FRONT: QUESTION / SCENARIO**

What environment variables and mock settings must you configure to redirect the official CDK `integ-runner` to run integration tests against a local LocalStack instance?

BACK: MECHANICAL EXPLANATION

Export the following environment variables to redirect the AWS SDK client calls of `integ-runner`:

- `AWS_ENDPOINT_URL="http://localhost.localstack.cloud:4566"` (resolves virtual-hosted S3 subdomains).
- `AWS_ENDPOINT_URL_S3="http://s3.localhost.localstack.cloud:4566"` (forces S3 client queries locally).
- `AWS_ACCESS_KEY_ID="test"` & `AWS_SECRET_ACCESS_KEY="test"` (mock credentials).
- `CDK_DEFAULT_ACCOUNT="000000000000"` (LocalStack default account ID).
- Run `!stk cdk bootstrap --force` before launching `integ-runner`.

CARD 14: CUSTOM RESOURCES VS AWS-CUSTOM-RESOURCE**FRONT: QUESTION / SCENARIO**

When would you implement a custom resource, and how does `AwsCustomResource` simplify custom resource deployments compared to a custom Lambda function?

BACK: MECHANICAL EXPLANATION

- ****When****: When you must execute logic during CloudFormation CREATE, UPDATE, or DELETE phases that isn't supported by standard CloudFormation or CDK constructs.
- ****`AwsCustomResource`****: If your custom logic requires only a single AWS SDK call, `AwsCustomResource` handles the boilerplate entirely. It deploys an automated singleton Lambda function that executes the specified SDK API call on your behalf at deploy time, removing the need to write, compile, and maintain a custom Lambda codebase.

CARD 15: THE ESCAPE HATCH PATTERN

FRONT: QUESTION / SCENARIO

What is an "Escape Hatch" in the AWS CDK, and when is it appropriate to use it? Show a typical code example.

BACK: MECHANICAL EXPLANATION

- **Definition**: An escape hatch allows you to bypass Level 2 or Level 3 construct abstractions to access and modify the low-level Level 1 (`Cfn*`) CloudFormation resources directly.
- **When**: Appropriate when an AWS service introduces a new feature or property that is supported in CloudFormation but not yet exposed in the high-level CDK APIs.

- **Example**:

```
const launchConfig = myAutoScalingGroup.node.findChild("LaunchConfig") as
CfnLaunchConfiguration;
launchConfig.metadataOptions = { httpPutResponseHopLimit: 2 };
```

CARD 16: ENVIRONMENT BOOTSTRAPPING

FRONT: QUESTION / SCENARIO

What is CDK Bootstrapping, what resources does it provision physically, and when must you run it again?

BACK: MECHANICAL EXPLANATION

- **Definition**: Bootstrapping is a one-time preparation step executed in every AWS account/region target environment (`cdk bootstrap`).
- **Resources**: It deploys a CloudFormation stack named `CDKToolkit` containing:
 - An S3 bucket for storing file assets (Lambda zip packages, templates).
 - An ECR repository for storing Docker image assets.
 - IAM roles giving the CDK CLI permission to deploy stacks in the account.
- **Re-bootstrap**: You must re-run bootstrapping when upgrading to new bootstrap template versions containing updated permission boundaries or feature capabilities (e.g. version 32 for `cdk orphan` and `cdk refactor`).

CARD 17: COMPOSITION VS INHERITANCE

FRONT: QUESTION / SCENARIO

Explain why CDK best practices favor Composition over Inheritance when building custom constructs.

BACK: MECHANICAL EXPLANATION

Inheriting from existing service-specific constructs (e.g., subclassing `s3.Bucket`` to enforce encryption) introduces tight coupling and can leak internal implementation changes.

- **Composition**: Always inherit from the base `Construct`` class to create custom components.
- **Structure**: Bundle the necessary resources as child components inside the constructor (e.g., instantiating a private S3 bucket and an SNS topic inside a custom `SecureDataStore`` construct). This clean encapsulation promotes isolation, prevents parent-class parameter pollution, and makes sharing across teams straightforward.

CARD 18: LOGICAL IDS VS PHYSICAL NAMES

FRONT: QUESTION / SCENARIO

What is the difference between a construct's logical ID and its physical resource name? What is the danger of hardcoding physical names?

BACK: MECHANICAL EXPLANATION

- **Logical ID**: An internal alphanumeric identifier (e.g., `MyTable794EDED1``) generated by the CDK based on the construct path hash. CloudFormation uses it to track the resource state.
- **Physical Name**: The actual name of the resource in your AWS account (e.g., `OrdersTable``).
- **Danger**: Hardcoding physical names (e.g., `tableName: 'MyBucket``) prevents deployment flexibility because you cannot deploy multiple environments (e.g. dev, prod) to the same AWS account without name collisions. It also blocks resource replacements during updates, causing deployments to fail.

CARD 19: POLICY COMPLIANCE AUDITING WITH CDK-NAG**FRONT: QUESTION / SCENARIO**

What is `cdk-nag`, and how does it utilize CDK Aspects to audit synthesized CloudFormation templates for compliance?

BACK: MECHANICAL EXPLANATION

- **`cdk-nag`**: An open-source auditing library that scans your synthesized templates for security vulnerabilities and compliance alignment (e.g. PCI DSS, HIPAA, NIST 800-53).
- **Aspect Integration**: It integrates as an Aspect on your application root scope. During synthesis, it traverses the entire construct tree (Visitor pattern), examining every node properties. It flags security gaps, prints warnings, and generates a structured audit report (`validation-report.json`) tracking suppressed and active policy violations.

CARD 20: CDK ASPECTS VISITOR PATTERN**FRONT: QUESTION / SCENARIO**

What are CDK Aspects, and what is a practical use case for applying them to a deployed stack?

BACK: MECHANICAL EXPLANATION

- **Definition**: Aspects are constructs that allow you to apply operations to all constructs within a given scope. Under the hood, they utilize the standard software engineering **Visitor pattern** to traverse every node in the synthesized construct tree.
- **Use Case**: Best for applying global policies or configurations dynamically. For example, you can write an Aspect that scans every resource in a test stack and automatically overrides their `RemovalPolicy` to `DESTROY`, ensuring that no orphaned DynamoDB tables or S3 buckets are left behind in your staging account.

CARD 21: CDK FEATURE FLAGS**FRONT: QUESTION / SCENARIO**

What are CDK Feature Flags, where are they defined, and how do they prevent breaking changes when upgrading the CDK CLI?

BACK: MECHANICAL EXPLANATION

- **Definition**: Feature flags are boolean flags declared in the ``context`` block of your ``cdk.json`` file.
- **Purpose**: They control whether the CDK Construct Library should use newer, updated behaviors or stick to older legacy defaults. This prevents breaking changes from affecting your existing deployed environments when you upgrade the CDK CLI version. Greenfield projects created with ``cdk init`` have all modern feature flags enabled by default, while legacy apps keep their original context values to maintain template stability.

CARD 22: CROSS-ACCOUNT CONSTRUCT REFERENCES**FRONT: QUESTION / SCENARIO**

How does the CDK handle resource referencing across different AWS accounts, and what are the limitations during refactoring?

BACK: MECHANICAL EXPLANATION

- **References**: Multiple stacks can be declared under an ``App``. When a stack in Account B references a resource from a stack in Account A, CDK automatically configures secure cross-account logical exports via the bootstrapped deployment roles, authorizing the reference.
- **Refactoring Constraint**: When executing ``cdk refactor`` or ``cdk orphan`` to move resources between stacks, **all** stacks involved must reside in the same environment (account and region). Resources cannot be programmatically moved across account boundaries because CloudFormation cannot physically migrate active physical resources between distinct AWS accounts.

CARD 23: ACCL_LOGS AND CUSTOM RESOURCE FAILURE DRIFT**FRONT: QUESTION / SCENARIO**

What debugging improvement did the CDK CLI introduce in June 2026 to assist when Custom Resources or ECS deployments fail?

BACK: MECHANICAL EXPLANATION

Historically, when an ECS Fargate deployment or a custom resource Lambda execution failed, developers had to open the AWS Console and search through CloudWatch log groups to discover the traceback error. In June 2026, the CDK CLI was updated to automatically fetch and print the relevant ECS container-level and custom resource Lambda failure logs directly in your terminal standard output upon deployment failure, radically speeding up the developer feedback loop.

CARD 24: CDK MIXINS AND THE WITH() METHOD**FRONT: QUESTION / SCENARIO**

What are CDK Mixins, and how do you apply them to constructs using the `.with()` method?

BACK: MECHANICAL EXPLANATION

- **Definition**: Mixins are highly composable, reusable helper packages that can be dynamically applied to any Level 1 or Level 2 construct. They are available inside the `mixins` namespace of their respective service modules (e.g., s3.mixins`).`
- **Usage**: You apply them to constructs using the `.with()` method (e.g., `new s3.CfnBucket(this, 'MyBucket').with(new s3.mixins.BucketVersioning())`).` This applies the targeted features programmatically to both L1 and L2 constructs, avoiding the need to extend classes.

CARD 25: CDK EXPLORE COMMAND**FRONT: QUESTION / SCENARIO**

What is the ``cdk explore`` command, and how does it assist developers in reviewing their synthesized Cloud Assembly?

BACK: MECHANICAL EXPLANATION

- **Command**: ``cdk explore`` spins up a local web server displaying a graphical user interface built on AWS Cloudscape.
- **Function**: It allows developers to browse stack resources, view synthesized CloudFormation templates, and navigate interactively between their source code files and generated JSON configurations. It features diagnostics that automatically refresh when ``cdk.out`` changes, as well as editor integrations (CodeLens) to pinpoint resource definitions instantly.

CARD 26: CDK MIGRATE COMMAND**FRONT: QUESTION / SCENARIO**

Explain how the ``cdk migrate`` command is used to modernize pre-existing cloud environments.

BACK: MECHANICAL EXPLANATION

- **``cdk migrate``**: A powerful CLI migration utility that translates pre-existing cloud infrastructure into fully typed, structured CDK code.
- **Mechanism**: It takes running physical AWS resources in your account, pre-existing CloudFormation stacks, or raw local JSON/YAML templates, analyzes their schemas, and programmatically generates the matching CDK application codebase (e.g. in TypeScript). This allows teams to quickly bring unmanaged legacy infrastructure under robust, version-controlled Infrastructure as Code.

CARD 27: THE PROGRAMMATIC TOOLKIT LIBRARY

FRONT: QUESTION / SCENARIO

When and why would you import `@aws-cdk/toolkit-lib` in Node.js instead of calling the standard `aws-cdk` CLI?

BACK: MECHANICAL EXPLANATION

- **Why**: When you must execute CDK lifecycle actions programmatically (e.g. building custom developer portals, internal platform portals, or automated testing orchestrators) instead of executing raw shell commands.
- **Usage**: By importing the programmatic `Toolkit` class directly in your Node.js application, you can orchestrate synthesis, deployments, and deletions dynamically:

```
const toolkit = new Toolkit();
const source = await toolkit.fromCdkApp("ts-node app.ts");
const assembly = await toolkit.synth(source);
await toolkit.deploy(assembly);
```

CARD 28: UNIT TESTING (SNAPSHOTS VS ASSERTIONS)

FRONT: QUESTION / SCENARIO

Compare the purpose, strengths, and weaknesses of Snapshot Testing versus Fine-Grained Assertions in AWS CDK applications.

BACK: MECHANICAL EXPLANATION

- **Fine-Grained Assertions**:
 - **Purpose**: Validates specific resource attributes (e.g., verifying that S3 bucket encryption is set to `S3_MANAGED`).
 - **Strength**: Highly stable and resilient. It evaluates only targeted properties and is unaffected by minor metadata adjustments. Great for test-driven development (TDD).
- **Snapshot Tests**:
 - **Purpose**: Compares the entire synthesized template against a baseline JSON master file.
 - **Strength**: Catches unexpected side effects and template drift immediately during refactoring.
 - **Weakness**: Highly fragile. Minor CLI metadata changes or dependency updates fail the test, forcing frequent snapshot updates.

CARD 29: REMOVALPOLICY AND DELETION LIFECYCLE

FRONT: QUESTION / SCENARIO

What is the function of the `RemovalPolicy`` property in the AWS CDK, and what are its defaults for stateful versus stateless resources?

BACK: MECHANICAL EXPLANATION

- **Function**: `RemovalPolicy`` determines whether the physical resource in AWS should be physically deleted (`RemovalPolicy.DESTROY``) or left intact (`RemovalPolicy.RETAIN``) when its managing Stack or parent construct is deleted.
- **Defaults**:
 - **Stateful Resources**: (DynamoDB Tables, S3 Buckets, RDS Databases) default to `RETAIN`` as a guardrail against accidental database deletion and data loss.
 - **Stateless Resources**: (IAM Roles, Security Groups, Lambda Functions) default to `DESTROY``, cleaning up seamlessly when the stack is removed.

CARD 30: THE CONSTRUCT TREE APP & STACK SCOPES

FRONT: QUESTION / SCENARIO

Explain why all AWS resource constructs must be child nodes of a `Stack`` construct, and how the `App`` construct fits into the CDK hierarchy.

BACK: MECHANICAL EXPLANATION

- **App**: The root of the CDK construct tree. It contains one or more Stacks and manages the overall synthesis lifecycle.
- **Stack**: The unit of deployment in AWS CloudFormation.
- **Resource Scope**: CloudFormation can only provision resources that are declared inside a specific stack template. Therefore, all physical resource constructs (e.g., Lambda functions, VPCs) must be instantiated as child nodes (directly or nested) of a `Stack`` instance. The Stack serializes these child nodes into its synthesized CloudFormation template.